Jenkins-challenge

Use the below code and implement our CICD stages.

- **Source Code:** https://github.com/betawins/hiring-app.git
    - Repository contains the application code for the hiring app.
- Create one Declarative pipeline job
- Create one Scripted pipeline job
- Create one multi stage pipeline job
- Create one parallel stage pipeline job

Step 1:
Runtime Environment: Java 21 (Amazon Corretto)
sudo dnf install java-21-amazon-corretto -y

Install git: Sudo systemctl -y install git

Steps: installed through jenkins official Site:
Step 2: Long Term Support release
sudo wget -O /etc/yum.repos.d/jenkins.repo \ https://pkg.jenkins.io/rpm-stable/jenkins.repo
sudo yum upgrade
Add required dependencies for the jenkins package
sudo yum install fontconfig java-21-openjdk
sudo yum install jenkins
sudo systemctl daemon-reload

Step 3: Maven
cd /opt/
Sudo wget https://dlcdn.apache.org/maven/maven-3/3.10.0/binaries/apache-maven-3.10.0
bin.tar.gz
sudo tar -xvf apache-maven-3.10.0-bin.tar.gz
Edit → vi .bash_profile

MVN_HOME=/opt/apache-maven-3.10.0/bin
PATH=$PATH:$HOME/bin:$MVN_HOME

Export PATH
Run: source ~/.bash_profile
Save; :wq!
check: echo $MVN_HOME
→ mvn –version

Configure Maven in Jenkins (Global Tool Configuration)

1. Go to **Manage Jenkins → Tools**
2. Scroll to **Maven installations → Add Maven**
3. Name: Maven-3.10
4. **Uncheck** "Install automatically" (since it's already on your agent)
5. MAVEN_HOME: /opt/apache-maven-3.10.0
6. **Save**

If you prefer auto-install, keep it checked and choose version 3.9.x or 3.10.x.

0.3 Add JDK (optional but recommended)

Manage Jenkins → Tools → JDK installations → Add JDK

- Name: JDK-21
- Uncheck auto-install
- JAVA_HOME: /usr/lib/jvm/java-21-amazon-corretto.x86_64

PIPELINE 1: Declarative Pipeline Job

1.1 Create the Job

1. Jenkins Dashboard → New Item
2. Name: hiring-app-declarative
3. Type: Pipeline → OK

1.2 Configure

- General → ✅ GitHub project → https://github.com/betawins/hiring-app.git
- Pipeline section:
  - Definition: Pipeline script
  - Paste the code below

Pipeline Script:

```
pipeline {
  agent { label 'agent-01' }

  tools {
    maven 'Maven-3.10'
    // jdk removed – JAVA_HOME set manually below
  }

  environment {
    JAVA_HOME = '/usr/lib/jvm/java-21-amazon-corretto.x86_64'
    PATH      = "${env.JAVA_HOME}/bin:${env.PATH}"
    APP_NAME  = 'hiring-app'
    GIT_URL   = 'https://github.com/betawins/hiring-app.git'
  }

  options {
    timestamps()
    buildDiscarder(logRotator(numToKeepStr: '10'))
    disableConcurrentBuilds()
  }

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: "${env.GIT_URL}"
      }
    }

    stage('Verify Environment') {
      steps {
        sh 'echo "JAVA_HOME=$JAVA_HOME"'
```

```
            sh 'which java && java -version'
            sh 'which mvn && mvn -version'
        }
    }

    stage('Build') {
        steps {
            sh 'mvn clean package -DskipTests'
        }
    }

    stage('Test') {
        steps {
            sh 'mvn test'
        }
        post {
            always {
                // allowEmptyResults: true -> do not fail if no XML test reports exist
                junit testResults: 'target/surefire-reports/*.xml', allowEmptyResults: true
            }
        }
    }

    stage('Archive Artifact') {
        steps {
            archiveArtifacts artifacts: 'target/*.war', fingerprint: true
        }
    }
  }

  post {
    success { echo "✅ ${env.APP_NAME} build SUCCESS" }
    failure { echo "❌ ${env.APP_NAME} build FAILED" }
    always  { cleanWs() }
  }
}
```

Run the Build

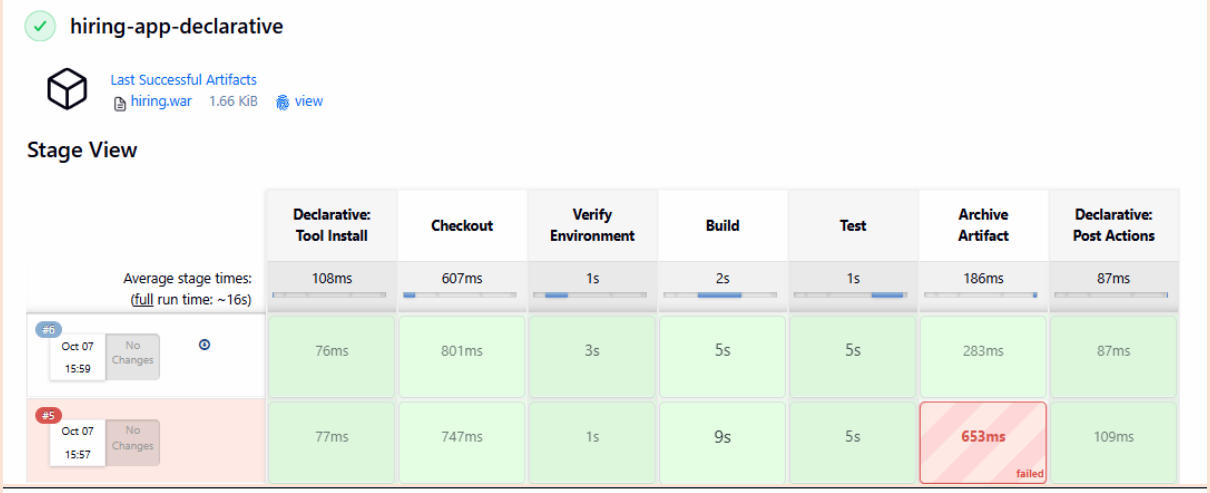
1. On the job page → click **Build Now**
2. Under **Build History** (left panel) → click build **#6** (or newest)
3. Click **Console Output**
4. ✅ Expected console tail (SUCCESS)

STEP 3: Verify the artifact was archived

After the build succeeds:

1. Click the build number (#6)
2. In the left menu, click **Artifacts** (only appears if artifacts were archived)
3. You should see: target/hiring.war
4. Click it to download → confirms the WAR is retrievable

Implementation:







🅰 **SCRIPTED PIPELINE — hiring-app-scripted**

Create job: New Item → Name hiring-app-scripted → **Pipeline** → OK

```
node('agent-01') {

    def appName = 'hiring-app'
    def gitUrl  = 'https://github.com/betawins/hiring-app.git'

    withEnv([
        'JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto.x86_64',
        'PATH+JAVA=/usr/lib/jvm/java-21-amazon-corretto.x86_64/bin',
        'PATH+MAVEN=/opt/apache-maven-3.10.0/bin'
    ]) {
        try {
            stage('Checkout') {
                echo "Cloning ${gitUrl}"
                git branch: 'main', url: gitUrl
            }

            stage('Verify Environment') {
                sh 'echo "JAVA_HOME=$JAVA_HOME"'
                sh 'which java && java -version'
                sh 'which mvn && mvn -version'
            }

            stage('Build') {
                sh 'mvn clean package -DskipTests'
            }

            stage('Test') {
                sh 'mvn test'
            }

            stage('Archive Artifact') {
                archiveArtifacts artifacts: 'target/*.war', fingerprint: true
                junit testResults: 'target/surefire-reports/*.xml', allowEmptyResults: true
            }

            echo "☑ ${appName} build SUCCESS"

        } catch (Exception e) {
            echo "❌ Build FAILED: ${e.message}"
            currentBuild.result = 'FAILURE'
            throw e

        } finally {
            cleanWs()
            echo "Workspace cleaned."
```

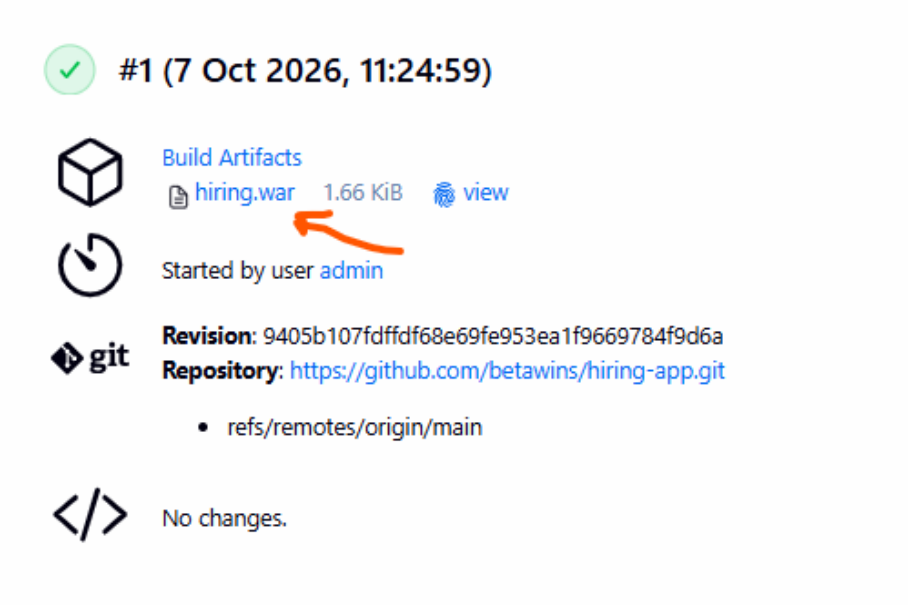
```
        }
    }
}
```

Key differences from Declarative:

- No tools { } block → Maven/JDK paths injected via withEnv
- Uses try/catch/finally instead of post { }
- Wrapped in node('agent-01') { }

Implementation:





MULTI-STAGE PIPELINE — hiring-app-multistage

Create job: New Item → Name hiring-app-multistage → **Pipeline** → OK

```
pipeline {
  agent { label 'agent-01' }

  tools {
    maven 'Maven-3.10'
  }

  environment {
    JAVA_HOME = '/usr/lib/jvm/java-21-amazon-corretto.x86_64'
    PATH      = "${env.JAVA_HOME}/bin:${env.PATH}"
    APP_NAME  = 'hiring-app'
    GIT_URL   = 'https://github.com/betawins/hiring-app.git'
    BUILD_DIR = 'target'
  }

  options {
    timestamps()
    buildDiscarder(logRotator(numToKeepStr: '10'))
    timeout(time: 30, unit: 'MINUTES')
  }

  stages {
    stage('1. Checkout') {
      steps {
        git branch: 'main', url: "${env.GIT_URL}"
      }
    }

    stage('2. Validate Env') {
      steps {
        sh 'echo "JAVA_HOME=$JAVA_HOME"'
        sh 'java -version'
        sh 'mvn -version'
      }
    }

    stage('3. Compile') {
      steps {
        sh 'mvn clean compile'
      }
    }

    stage('4. Unit Test') {
      steps {
        sh 'mvn test'
      }
```

```
        post {
            always {
                junit testResults: 'target/surefire-reports/*.xml', allowEmptyResults: true
            }
        }
    }

    stage('5. Package') {
        steps {
            sh 'mvn package -DskipTests'
        }
    }

    stage('6. Code Quality') {
        steps {
            echo 'Running static checks (placeholder)'
            sh 'ls -la target/'
        }
    }

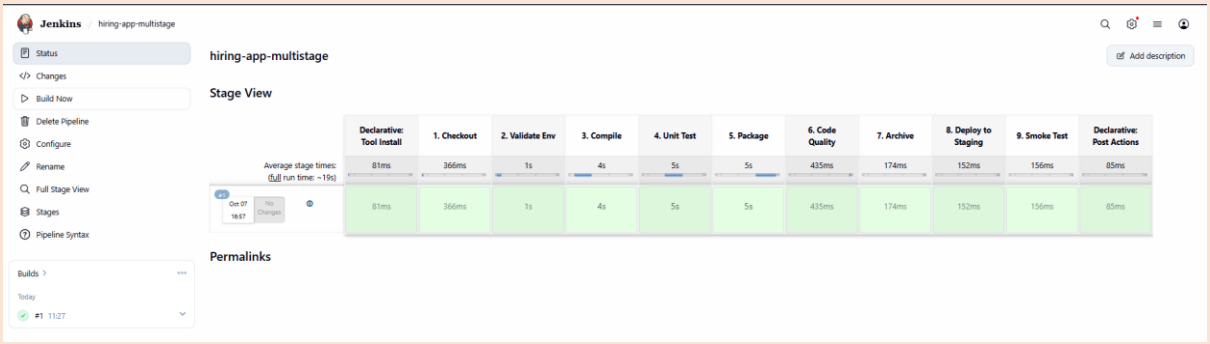
    stage('7. Archive') {
        steps {
            archiveArtifacts artifacts: "${env.BUILD_DIR}/*.war", fingerprint: true
        }
    }

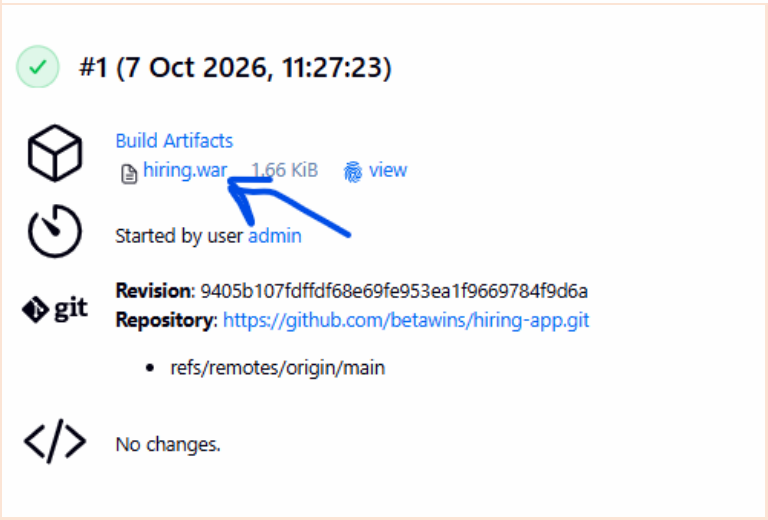
    stage('8. Deploy to Staging') {
        steps {
            echo "Deploying ${env.APP_NAME} to Staging (simulated)"
            // Real: scp target/*.war user@staging:/opt/tomcat/webapps/
        }
    }

    stage('9. Smoke Test') {
        steps {
            echo 'Running smoke tests on Staging (simulated)'
        }
    }
  }

  post {
    success { echo "☑ Full pipeline SUCCESS for ${env.APP_NAME}" }
    failure { echo "❌ Pipeline FAILED at some stage" }
    unstable { echo "⚠  Pipeline UNSTABLE" }
    always  { cleanWs() }
  }
}
```

Implementation:





PARALLEL PIPELINE — hiring-app-parallel

Create job: New Item → Name hiring-app-parallel → Pipeline → OK

```
pipeline {
  agent { label 'agent-01' }

  tools {
    maven 'Maven-3.10'
  }

  environment {
    JAVA_HOME = '/usr/lib/jvm/java-21-amazon-corretto.x86_64'
    PATH      = "${env.JAVA_HOME}/bin:${env.PATH}"
    GIT_URL   = 'https://github.com/betawins/hiring-app.git'
  }

  options {
    timestamps()
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: "${env.GIT_URL}"
```

```
    }
}

stage('Build') {
    steps {
        sh 'mvn clean compile'
    }
}

// ===== PARALLEL STAGES =====
stage('Parallel Quality Gates') {
    parallel {

        stage('Unit Tests') {
            steps {
                echo '▶ Running Unit Tests'
                sh 'mvn test'
            }
            post {
                always {
                    junit testResults: 'target/surefire-reports/*.xml', allowEmptyResults: true
                }
            }
        }

        stage('Static Code Analysis') {
            steps {
                echo '▶ Running Static Code Analysis (simulated)'
                sh 'echo "Checkstyle/PMD placeholder"; ls -la'
            }
        }

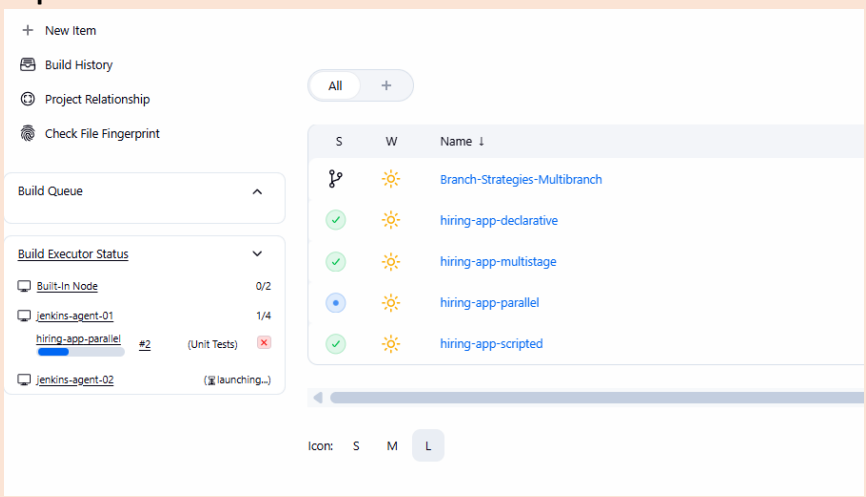
        stage('Dependency Check') {
            steps {
                echo '▶ Checking dependencies'
                sh 'mvn dependency:tree'
            }
        }

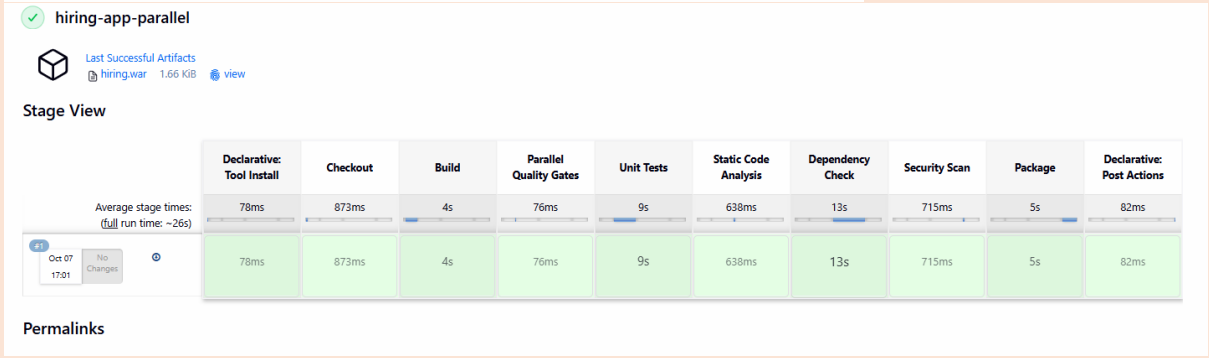
        stage('Security Scan') {
            steps {
                echo '▶ Running Security Scan (simulated)'
                sh 'echo "OWASP Dependency Check placeholder"'
            }
        }
    }
}
```

```
        stage('Package') {
            steps {
                sh 'mvn package -DskipTests'
                archiveArtifacts artifacts: 'target/*.war', fingerprint: true
            }
        }
    }

    post {
        success { echo "✅ Parallel pipeline SUCCESS" }
        failure { echo "❌ Parallel pipeline FAILED" }
        always  { cleanWs() }
    }
}
```

Implementation:







Jenkins-challenge